shieldify



SMART CONTRACT SECURITY ASSESSMENT

Rabtools

SECURITY REVIEW

21 OCTOBER 2026

rt contracts, our shielding · Your smart contracts, our shielding · Your smart contracts, our shielding · Your smart contracts

Contents

1 **About Shieldify** 3

2 **Disclaimer** 3

3 **About Rabtools** 3

4 **Risk Classification** 4

4.1 Impact 4

4.2 Likelihood 4

5 **Security Review Summary** 4

5.1 Protocol Summary 5

5.2 Scope 5

6 **Findings Summary** 5

7 **Findings** 7

1 About Shieldify

Positioned as the first hybrid Web3 Security company, Shieldify shakes things up with a unique subscription-based auditing model that entitles the customer to unlimited audits within its duration, as well as top-notch service quality thanks to a disruptive 6-layered security approach. The company works with very well-established researchers in the space and has secured multiple millions in TVL across protocols, also can audit codebases written in Solidity, Vyper, Rust, Cairo, Move and Go.

Learn more about us at shieldify.org.

2 Disclaimer

This security review does not guarantee bulletproof protection against a hack or exploit. Smart contracts are a novel technological feat with many known and unknown risks. The protocol, which this report is intended for, indemnifies Shieldify Security against any responsibility for any misbehavior, bugs, or exploits affecting the audited code during any part of the project's life cycle. It is also pivotal to acknowledge that modifications made to the audited code, including fixes for the issues described in this report, may introduce new problems and necessitate additional auditing.

3 About Rabtools

Rabtools is a treasury management system on Avalanche C-Chain that automates reward claiming and compounding for Pharaoh AutoVault positions. Each user deploys a personal treasury contract through a factory, funds it with PHAR, xPHAR, P33, or AVAX, and selects a reward asset (USDC, WAVAX, or BTC.b) in which the AutoVault pays out its rewards.

At a high level, the main functionalities are:

- Treasury deployment through a factory, with a creation fee and an owner-defined payout wallet and reward asset
- Conversion of PHAR to xPHAR, staking of xPHAR into the Pharaoh AutoVault, and wrapping or redeeming xPHAR through the P33 ERC-4626 vault
- Reward claiming that pays the payout wallet its share immediately and retains a configurable portion for compounding
- Auto-compounding of retained rewards by swapping them to PHAR through an aggregator, converting to xPHAR, and restaking into the AutoVault, with a 0.33% protocol fee

- A per-treasury automator (bot wallet) appointed by the owner that triggers claims and compounding off-chain on an epoch schedule
- Owner withdrawal functions for each asset class, plus a bundled `fullWithdraw()` exit and, in TreasuryV4, an ERC-20 recovery function

4 Risk Classification

SEVERITY	IMPACT: HIGH	IMPACT: MEDIUM	IMPACT: LOW
Likelihood: High	CRITICAL	HIGH	MEDIUM
Likelihood: Medium	HIGH	MEDIUM	LOW
Likelihood: Low	MEDIUM	LOW	LOW

4.1 Impact

- **High** – results in a significant risk for the protocol's overall well-being. Affects all or most users
- **Medium** – results in a non-critical risk for the protocol affects all or only a subset of users, but is still unacceptable
- **Low** – losses will be limited but bearable, and covers vectors similar to griefing attacks that can be easily repaired

4.2 Likelihood

- **High** – almost certain to happen and highly lucrative for execution by malicious actors
- **Medium** – still relatively likely, although only conditionally possible
- **Low** – requires a unique set of circumstances and poses non-lucrative cost-of-execution to rewards ratio for the actor

5 Security Review Summary

The security review lasted 3 days with a total of 48 hours dedicated to the audit by the Shieldify team.

Overall, the code is well-written. The audit report contributed by identifying four Low and eight Informational severity issues. They're mainly related to reward-asset configuration and claim accounting, missing output and minimum checks around the P33 and AutoVault integrations, token recovery compatibility, and the trust and availability assumptions placed on the automator role and external Pharaoh dependencies.

The Rabtools team has done a great job and provided support and responses to all of the questions that the Shieldify researchers had.

5.1 Protocol Summary

PROJECT NAME	RABTOOLS
Repository	rabengine
Type of Project	DeFi Treasury Automation (Reward Claiming & Auto-Compounding)
Security Review Timeline	3 days
Review Commit Hash	453143767d48566b8878f27cc853441f9b225029
Fixes Review Commit Hash	3117c5456a108efff55ead968a8dfac358ac87c3

5.2 Scope

The following smart contracts were in the scope of the security review:

FILE	NSLOC
contracts/TreasuryV3.sol	283
contracts/TreasuryV4.sol	24
contracts/TreasuryFactoryV3.sol	47
contracts/TreasuryFactoryV4.sol	46
Total	400

6 Findings Summary

The following number of issues have been identified, sorted by their severity:

- **Low** issues: 4
- **Info** issues: 8

ID	TITLE	SEVERITY	STATUS
[L-01]	`setRewardAsset()` Changes the Measured Token but Not the Vault's Payout Token, so Claims Record Zero While Rewards Keep Arriving	LOW	ACKNOWLEDGED
[L-02]	Lack of Slippage Protection While Interacting With P33 Token	LOW	ACKNOWLEDGED

ID	TITLE	SEVERITY	STATUS
[L-03]	Compounds Below the Vault's 1e18 Minimum Revert Whole, and the Keeper Submits Them Without Simulating	LOW	ACKNOWLEDGED
[L-04]	TreasuryV4 Cannot Recover Tokens Whose Transfer Returns No Data	LOW	FIXED
[I-01]	fullWithdraw Exits Without Claiming Accrued Vault Rewards	INFO	ACKNOWLEDGED
[I-02]	`lastCompoundPortion` Is Written Three Times and Read Nowhere On-Chain, and Manual and Automated Compounding Size Themselves Differently	INFO	FIXED
[I-03]	`fullWithdraw()` Wraps Any Positive xPHAR Balance Into P33 Without Checking the Shares It Mints	INFO	ACKNOWLEDGED
[I-04]	rewardAsset Is Checked for Nonzero and Nothing Else	INFO	ACKNOWLEDGED
[I-05]	Treasury xPHAR Moves One Way, and XPhar's Pause Stops Conversions but Not Transfers	INFO	ACKNOWLEDGED
[I-06]	Claim and Exit Timing Across the Fleet Depends on the `AutoVault` Operator Role	INFO	ACKNOWLEDGED
[I-07]	Trusted Automator Controls the Compounding Target, Calldata, and Reward-Token Spending Approval	INFO	ACKNOWLEDGED
[I-08]	One Wei of AVAX Permanently Locks a Contract-Owned Treasury Out of `fullWithdraw()`	INFO	FIXED

7 Findings

L-01

LOW ACKNOWLEDGED

`setRewardAsset()` Changes the Measured Token but Not the Vault's Payout Token, so Claims Record Zero While Rewards Keep Arriving

Description

The treasury stores its reward asset in one variable and uses it for three jobs. It names the output token when depositing into the AutoVault, which fixes the vault's per-account payout token at deposit time. It names the token whose balance delta `claimRewards()` measures around `claim()`. And it names the swap input for `autoCompound()`. `setRewardAsset()` rewrites the variable on the spot. The vault's stored preference does not move until the next deposit of at least 1e18 wei switches it. The vault itself exposes a preference setter (`AutoVault3.sol`), but the treasury has no wrapper for it.

While the two settings disagree, a successful vault claim transfers the old token while the treasury measures a zero increase in the new one. The delta is zero, the early-return branch fires, `lastCompoundPortion` resets, and the log shows `RewardsClaimed(0, 0, 0, ...)`. Nothing distinguishes that from a quiet week, and the payout wallet, which the whitepaper describes as receiving its share on every claim (`client/src/pages/whitepaper.tsx`), receives nothing from that claim. The old-token proceeds stay directly held by the treasury. Switching the vault's output, by deposit or by the vault's own setter, settles pending old-token rewards at the vault level (`AutoVault3.sol`), again outside the treasury's claim measurement.

Restoring the setting is not a recovery. A later claim measures only the delta from that moment, so new accruals pay out while the earlier balance stays held, and V3 has no general route to retrieve it: `fullWithdraw()` sweeps PHAR, P33, and AVAX only, and no other function moves a held reward token to the owner. For the payout omission to bite, the split must actually assign a payout share, meaning compounding is disabled or set below 100 percent.

Location of Affected Code

File: contracts/TreasuryV3.sol#L175-L183

```
uint256 rewardBefore = IERC20(rewardAsset).balanceOf(address(this));
IAutoVault(autoVault).claim();
uint256 rewardAfter = IERC20(rewardAsset).balanceOf(address(this));
uint256 rewardAmount = rewardAfter - rewardBefore;

if (rewardAmount == 0) {
  lastCompoundPortion = 0;
  emit RewardsClaimed(0, 0, 0, rewardAsset, payoutWallet);
```

```
    return;
  }
```

File: contracts/TreasuryV3.sol#L345-L350

```
function setRewardAsset(address _asset) external onlyOwner {
    require(_asset != address(0), "Invalid reward asset");
    address old = rewardAsset;
    rewardAsset = _asset;
    emit RewardAssetUpdated(old, _asset);
}
```

Impact

The payout wallet, which can be a distinct counterparty, has an entire claim's income withheld while the desync lasts, with events that read as healthy. The owner controls the configuration, both tokens can be perfectly supported outputs, and V4 can recover a held balance, though recovery retrieves the funds without repairing the claim accounting that misrouted them, which caps this at Low. Nothing signals any of it. Detection means balance forensics, not logs.

Recommendation

Make the switch atomic: settle and distribute old-token rewards, synchronize the vault preference, then update the local setting, in one owner operation. Handle the zero-stake case, since the vault's preference setter requires a positive balance. Expose the vault's actual output token and flag mismatches before claims pay out zeros, and add a separate owner-controlled recovery path for balances already held, because delta measurement cannot distribute them by itself.

Team Response

Acknowledged.

L-02

LOW ACKNOWLEDGED

Lack of Slippage Protection While Interacting With P33 Token

Description

The `TreasuryV3` contract interacts mainly with `AutoVault`, `p33Token`, and `xPHAR`. Depositing xPHAR tokens into `AutoVault` and redeeming will always give 1:1 xPHAR tokens, since the depositor is rewarded with extra rewards from the `AutoVault` contract. The PHAR-to-xPHAR conversion is treated as 1:1 by `convertEmissionsToken()`.

Slippage protection is needed for the P33 token operations because their output depends on the vault's current share-to-asset exchange rate. That rate can change between when the transaction is prepared and when it executes, so the same input can return fewer xPHAR tokens on redemption or fewer P33 shares on deposit.

Location of Affected Code

File: contracts/TreasuryV3.sol#L274-L286

```
function convertXPHARToP33(uint256 xpharAmount) external onlyOwner nonReentrant {
    // code
    IERC20(xpharToken).approve(p33Token, xpharAmount);
    uint256 p33Before = IERC20(p33Token).balanceOf(address(this));
    // @audit - no slippage check here
@>  P33(p33Token).deposit(xpharAmount, address(this));
    uint256 p33After = IERC20(p33Token).balanceOf(address(this));
    uint256 p33Received = p33After - p33Before;

    emit XPHARConvertedToP33(xpharAmount, p33Received);
}
```

File: contracts/TreasuryV3.sol#L127-L143

```
function depositP33(uint256 amount) external onlyOwner nonReentrant {
    // code
    uint256 xpharBefore = IERC20(xpharToken).balanceOf(address(this));
    // @audit - no slippage check here
    IP33(p33Token).redeem(amount, address(this), address(this));
    uint256 xpharAfter = IERC20(xpharToken).balanceOf(address(this));
    // code
}
```

Impact

The treasury may receive fewer xPHAR tokens when redeeming P33, or fewer P33 shares when depositing xPHAR, as the transaction can be sandwiched to manipulate the P33 token contract's asset-to-share conversion rate.

Recommendation

Add a `minXPHARReceived` parameter to `depositP33()` and a `minP33Received` parameter to `convertXPHARToP33()`. After each P33 redeem or deposit, compare the measured balance delta with the caller's minimum and revert if the output is lower.

Team Response

Acknowledged.

L-03

LOW ACKNOWLEDGED

Compounds Below the Vault's 1e18 Minimum Revert Whole, and the Keeper Submits Them Without Simulating

Description

The last leg of `autoCompound()` deposits the converted xPHAR into the AutoVault, which requires at least 1e18 wei (the deployed `AutoVault3` implementation). The treasury checks only that its PHAR and xPHAR deltas are positive (`TreasuryV3.sol`), never that the output reaches the vault minimum before calling `deposit()`. A compound whose output falls short reverts whole, taking the fee transfer, the swap, and the conversion with it. The reward asset then waits until the accumulated balance is large enough for one compound's output to clear the bar.

The keeper does not catch this. Its eligibility check is an input-token threshold (`server/automation.ts`), and it submits with a fixed gas limit without first simulating the treasury call, even though the whitepaper (`client/src/pages/whitepaper.tsx`) and `replit.md` both describe server-side simulation before submission. A quote that clears the input threshold but produces under 1e18 xPHAR becomes a mined revert.

One boundary worth drawing: the revert rolls back the fee, swap, and conversion of that transaction, while the claim and payout that preceded it were separate transactions and stand completed. The cost is failed-transaction gas and compounding that stays inert, not lost rewards.

Location of Affected Code

File: contracts/TreasuryV3.sol#L234-L241

```
IERC20(pharToken).approve(xpharToken, pharReceived);
uint256 xpharBefore = IERC20(xpharToken).balanceOf(address(this));
IxPHAR(xpharToken).convertEmissionsToken(pharReceived);
uint256 xpharReceived = IERC20(xpharToken).balanceOf(address(this)) - xpharBefore;
require(xpharReceived > 0, "No xPHAR received from conversion");

IERC20(xpharToken).approve(autoVault, xpharReceived);
IAutoVault(autoVault).deposit(xpharReceived, rewardAsset);
```

Impact

Compounding is inert for new and small treasuries and for any cycle whose market output lands under the minimum, and the automation loop spends gas on deterministic reverts, a cost the operator's bot pays. Rewards remain held and claimable throughout, which keeps this Low.

Recommendation

Check the expected output against the vault minimum before swapping and skip cleanly, carrying the reward asset, or the converted xPHAR, until a cycle clears 1e18. On the keeper side, simulate the treasury call before submission, which the documentation already promises.

Team Response

Acknowledged.

L-04

LOW FIXED

TreasuryV4 Cannot Recover Tokens Whose Transfer Returns No Data

Description

TreasuryV4's recovery function is the designated route for the balances the bundled exit does not sweep, and it is the version's answer to the missing reward-token withdrawal. It wraps the transfer in a typed call whose interface declares a `bool` return and requires that value to be true.

Two token families fail against that shape. Tokens whose `transfer()` returns no data, the older non-standard line, make the return-data decode revert before the check even evaluates. Tokens that return an explicit `false` fail the check itself. Either way, the one function built to retrieve stranded balances rejects the token, and no other function in the contract moves an arbitrary ERC-20 to the owner. The three documented reward outputs, USDC, WAVAX, and BTC.b, return booleans and recover normally. The exposure is the long tail of older or non-standard tokens a treasury can hold, which is exactly the population the hatch exists for.

Location of Affected Code

File: contracts/TreasuryV4.sol#L25-L35

```
function recoverToken(address token, uint256 amount) external onlyOwner nonReentrant {
  require(token != address(0), "Invalid token");
  require(amount > 0, "Amount must be greater than 0");
  require(
    IERC20TreasuryV4(token).balanceOf(address(this)) >= amount,
```

```
    "Insufficient token balance"
  );
  require(
    IERC20TreasuryV4(token).transfer(owner, amount), // typed call: decodes a required bool
    "Token transfer failed"
  );
```

Impact

For a V4 treasury holding such a token, the balance has no exit at all: the recovery transaction reverts and nothing else in the contract reaches the token. The exposure is bounded by which tokens a treasury actually holds, and the V4 factory has no deployments today, which keeps this a narrow Low ahead of any V4 rollout.

Recommendation

Route recovery through a reviewed safe-transfer helper with `SafeERC20` semantics: accept a call that succeeds and returns no data, reject an explicit `false` or a revert. Keep the ownership, amount, and reentrancy checks unchanged. This widens the hatch for quiet tokens; it does not bypass issuer freezes or the xPHAR transfer permissions, and the recovery documentation should not imply otherwise.

Team Response

Fixed.

I-01

INFO ACKNOWLEDGED

fullWithdraw Exits Without Claiming Accrued Vault Rewards

Description

`fullWithdraw()` withdraws the whole staked balance and never calls `claim()`. The rewards are preserved: the AutoVault checkpoints accruals into a per-account `storedRewards` on balance updates, keeps that record through a withdraw to zero, and lets `claim()` run with no stake behind it (the deployed `AutoVault3` implementation). The treasury's `pendingRewards()` view reads the vault's `earned()`, which includes the stored amount (`TreasuryV3.sol`), and the frontend displays it (`client/src/hooks/use-contracts.ts`).

So the exit is one transaction short, not lossy. The gap is operational: the backend's claim preflight skips zero-stake treasuries (`server/automation.ts`), so after a bundled withdrawal nothing automated picks the rewards up. The owner has to know that a separate manual `claimRewards()` remains, against a treasury the UI now presents as unstaked.

Location of Affected Code

File: contracts/TreasuryV3.sol#L296-L301

```
function fullWithdraw() external onlyOwner nonReentrant {
  uint256 stakedInVault = IAutoVault(autoVault).balanceOf(address(this));
  if (stakedInVault > 0) {
  IAutoVault(autoVault).withdraw(stakedInVault);
  emit UnstakedFromAutoVault(stakedInVault);
}
```

Impact

Final-epoch rewards wait in the vault until a separate claim runs. They stay visible and claimable throughout, so the cost is a missed step in the documented "full" exit, not value at risk.

Recommendation

Settle rewards as part of the exit, with one caveat the fix has to carry: a claim retains the compound portion, so even claim-then-withdraw leaves that balance behind unless a recovery step accompanies it. Spell out in the exit flow what remains outstanding and how to retrieve it.

Team Response

Acknowledged.

I-02

INFO FIXED

`lastCompoundPortion` Is Written Three Times and Read Nowhere On-Chain, and Manual and Automated Compounding Size Themselves Differently

Description

`claimRewards()` computes the split, pays the payout wallet its share at once, and records the retained portion in `lastCompoundPortion`. Three branches write it; a fourth clears it after compounding. The contract never reads it back. `autoCompound()` bounds its input only against the live balance of the reward asset, so the amount it compounds is whatever the caller says, up to everything the contract holds.

Two consumers read the variable off-chain, and they disagree. The frontend's manual-compound flow reads the getter to size the transaction and refuses to proceed on zero (`client/src/hooks/use-contracts.ts`). The backend reads the full reward balance instead (`server/automation.ts`). The snapshot describes only the latest claim, and any later claim overwrites it, including a zero-delta claim that writes zero. A treasury that retained rewards from an earlier claim, then had a quiet claim, shows the manual UI nothing while the contract would happily compound the whole balance and the backend intends to.

The payout side is safe: that value leaves the contract inside `claimRewards()` itself, before any compounding can reach it. Donated or residue balances can be swept through the compound path by the same authorized callers, which is an authority question rather than an accounting one. What is left here is an accounting design that does not say what it is for: one number, one writer on-chain, and two readers using different amounts.

Location of Affected Code

File: contracts/TreasuryV3.sol#L193-L200

```
uint256 compoundPortion = (rewardAmount * autoCompoundPercent) / 100;
uint256 payoutAmount = rewardAmount - compoundPortion;

if (payoutAmount > 0) {
 require(IERC20(rewardAsset).transfer(payoutWallet, payoutAmount), "Payout transfer failed");
}

lastCompoundPortion = compoundPortion;
```

File: contracts/TreasuryV3.sol#L214-L215

```
uint256 rewardBal = IERC20(rewardAsset).balanceOf(address(this));
require(rewardBal >= compoundGross, "Insufficient reward asset balance");
```

Impact

The manual and automated paths can compound different amounts from the same treasury, and the manual path can omit funds the treasury actually holds. No funds are lost and no authority is gained; the defect is that the on-chain record does not describe what either consumer does with it.

Recommendation

Pick one accounting policy and document it. If automation should spend only explicitly retained allocations, maintain a token-specific accumulator that adds each claim's portion and subtracts successful compounds, and define what happens on configuration changes. Capping against the latest snapshot will not work, since new claims overwrite it. If balance-based compounding is intended, use the balance consistently in the UI and explain what the snapshot represents.

Team Response

Fixed.

I-03

INFO ACKNOWLEDGED

`fullWithdraw()` Wraps Any Positive xPHAR Balance Into P33 Without Checking the Shares It Mints

Description

The xPHAR leg of `fullWithdraw()` runs whenever the balance is nonzero and never asks how many shares came back; the manual `convertXPHARToP33()` shares the missing postcondition. P33 is an ERC-4626 vault over xPHAR, and its deposit takes the assets in and stakes them into the vote module before minting the computed share amount (the deployed `P33` vault). If the applicable share conversion rounds a small input down to zero, a deposit can transfer the xPHAR without minting anything to the treasury.

Whether a given amount rounds to zero depends on the live share ratio and the imported ERC-4626 implementation; a single wei does not yield zero shares at every ratio, unsolicited xPHAR reaching the treasury at all depends on the token's transfer whitelist, and the deposit itself must clear P33's execution gates for the transfer to happen. What is certain is the shape: the treasury hands over any positive balance without requiring a positive share return, on the bundled exit and on the manual conversion alike.

Location of Affected Code

File: contracts/TreasuryV3.sol#L303-L310

```
uint256 xpharBal = IERC20(xpharToken).balanceOf(address(this));
if (xpharBal > 0) {
  IERC20(xpharToken).approve(p33Token, xpharBal);
  uint256 p33Before = IERC20(p33Token).balanceOf(address(this));
  IP33(p33Token).deposit(xpharBal, address(this));
  uint256 p33After = IERC20(p33Token).balanceOf(address(this));
```

```
    emit XPHARConvertedToP33(xpharBal, p33After - p33Before);
}
```

Impact

Dust-scale loss when the rounding condition is met, once per exit or manual conversion that carries residue. No mechanism to scale it into material loss was found.

Recommendation

Quote the shares first with `convertToShares()` and require a positive mint, or skip below a stated dust threshold, leaving sub-share amounts as loose xPHAR. Apply the same check to the manual conversion.

Team Response

Acknowledged.

I-04

INFO ACKNOWLEDGED

rewardAsset Is Checked for Nonzero and Nothing Else

Description

Factory and setter alike validate a single property of the reward asset: not the zero address. The AutoVault enforces its output whitelist at deposit time (the deployed `AutoVault3` implementation), not at claim time, so a bad choice surfaces piecemeal. An off-whitelist token makes every treasury deposit revert while claims keep paying under whatever preference the vault already holds. A non-contract address fails the `balanceOf()` calls in `claimRewards()` and `autoCompound()`, though `pendingRewards()`, a direct vault query, keeps answering. The frontend's supported-selection list does not stop a direct contract call from configuring anything else.

The misconfiguration is owner-selected and one transaction away from repair. Rewards that already landed under a mismatched setting must be recovered separately, and the bundled exit does not sweep them. That keeps this at informational. The cost is real all the same: a treasury can be created non-operational, after its 3 AVAX fee is spent, and stay that way until someone notices.

Location of Affected Code

File: contracts/TreasuryFactoryV3.sol#L26-L28

```
require(payoutWallet != address(0), "Invalid payout wallet");
require(rewardAsset != address(0), "Invalid reward asset");
require(msg.value >= CREATION_FEE, "Must send 3 AVAX creation fee");
```

Impact

Owner-inflicted, recoverable downtime of the reward pipeline, with the creation fee already sunk. No third party is harmed.

Recommendation

Check the token against the vault's output list at creation and on change, and require a contract that answers the ERC-20 views. A preflight view reporting whether the configured asset is payable by the vault would let owners and the backend catch dead configurations before funding.

Team Response

Acknowledged.

I-05

INFO ACKNOWLEDGED

Treasury xPHAR Moves One Way, and XPhar's Pause Stops Conversions but Not Transfers

Description

xPHAR restricts transfers through an exemption scheme (the deployed `XPhar` token) and pauses conversion and exit separately. Neither control is a treasury bug; both change what the in-scope code can do, so they belong in the operating picture.

The permission rule is one-sided, in the useful direction: `_isExempted()` accepts an exempt sender or an exemptTo destination, with a gauge and distributor exception besides, so a transfer needs one qualifying end, not two. The pause gates `convertEmissionsToken()` and `exit()` and leaves `_update()`, the transfer path, alone. During a pause, `convertPHARToXPHAR()` and the conversion leg of `autoCompound()` revert, PHAR cannot become xPHAR anywhere in the product, and staking of existing xPHAR keeps working. An exit that wraps already-held xPHAR into P33 does not call the paused functions and can remain available, subject to the whitelist and the external windows. On the window side, P33's withdrawal path skips the deposit-side lock modifier (`P33.sol`), and an on-chain check on 15 October 2026 found the deployed P33 and AutoVault exempt from the vote module's cooldown, so the weekly windows, not the cooldown, were the operative gates as of that date.

Location of Affected Code

File: contracts/TreasuryV3.sol#L151-L153

```
IERC20(pharToken).approve(xpharToken, pharAmount);
uint256 xpharBefore = IERC20(xpharToken).balanceOf(address(this));
IxPHAR(xpharToken).convertEmissionsToken(pharAmount);
```

Impact

External governance actions change which doors are open, on their schedule, with no signal through the treasury's own interface. No unauthorized action is involved; this is a dependency to disclose and monitor.

Recommendation

Keep the dependency chain, meaning the xPHAR whitelist, the pause semantics, the P33 windows, and the vault locks, in the operating runbook, and monitor the token's pause state and exemption lists alongside the fleet.

Team Response

Acknowledged.

I-06

INFO ACKNOWLEDGED

Claim and Exit Timing Across the Fleet Depends on the `AutoVault` Operator Role

Description

The whitepaper tells users they may withdraw from the AutoVault at any time (`client/src/pages/whitepaper.tsx`). The vault the treasuries actually use applies an unlocked-period requirement to withdrawals, claims, and deposits alike (the deployed `AutoVault3` implementation), and its operator or access hub can lock the current period at will and unlock it again. The treasury offers no alternative emergency path: `unstakeFromAutoVault()` and `fullWithdraw()` both go through the ordinary withdrawal. So exit, claim, and compound availability across the fleet ride on one external role and its schedule, and the product's blanket assurance is broader than the dependency permits.

This is not a hidden state. The vault exposes `isUnlocked()` and `OPERATOR()` and emits events on the transitions, so the fleet's availability is readable on-chain by anyone who looks. The treasuries just do not look, and the documentation does not mention the window at all.

Location of Affected Code

File: contracts/TreasuryV3.sol#L176

```
// reward availability set by operator swap scheduling
IAutoVault(autoVault).claim();
```

File: contracts/TreasuryV3.sol#L299

```
// exit window set by operator lock and unlock
IAutoVault(autoVault).withdraw(stakedInVault);
```

Impact

A scheduled or deliberate operator lock stops claims, compounds, and exits fleet-wide until unlocked. No unauthorized action is asserted; the gap is between a documented promise and the dependency's actual behavior.

Recommendation

Correct the withdrawal-at-any-time claim to describe the weekly window and the operator's relock ability. Monitor the vault's lock events and alert on relocks that break the expected calendar.

Team Response

Acknowledged.

I-07

INFO ACKNOWLEDGED

Trusted Automator Controls the Compounding Target, Calldata, and Reward-Token Spending Approval

Description

The team's clarification for this audit states that automation and the automator role are trusted: a bot wallet is created for each treasury, appointed by the owner, to trigger claims and compounding. Under that scope, both behaviors described below require a malicious or compromised appointed automator, or a transaction the owner signed. No path allowing an unauthorized caller to invoke these operations has been demonstrated, so the finding stands as a disclosure of the role's actual authority rather than an exploitable defect.

What the code permits is worth stating precisely. `autoCompound()` takes its amount, target, and calldata from the authorized caller. The amount is bounded only by the live reward-token balance; `lastCompoundPortion` is not an internal spending limit. The fee, `floor(gross * 33 / 10000)`, goes to the fixed fee recipient, and the remainder is approved to the caller-selected address, which then receives the caller-selected calldata through an ordinary external call from the treasury: not a delegatecall, and no native value attached. The approval is reset after the call, which limits residual allowance but cannot undo spending that happened during it. The postconditions are a positive PHAR delta, a successful conversion, and a vault deposit; positive deltas prove movement, not a fair exchange. A later revert rolls back that transaction's effects, while earlier claim and payout transactions stand completed.

Two behaviors follow from that authority. The appointed caller can select a target that takes the approved reward amount and supplies PHAR in return. With the saved dependency behavior, at least 1e18 PHAR base units convert one-for-one into enough xPHAR to meet the vault minimum. For a gross balance `R`, the recipient can receive `R - floor(R * 33 / 10000)` reward units while the supplied PHAR becomes a new treasury stake. Whether that profits requires comparing the market value of the rewards against the PHAR supplied and the gas; subtracting quantities of two different tokens is not a profit calculation. The route needs an unpaused conversion, an accepting vault and output token, and the required transfer permissions.

The caller can also point the target at the xPHAR token and encode `exit(uint256)`. The token sees the treasury as its caller and burns the treasury's loose xPHAR under the rounded-up 50 percent penalty, capped by the available reserve. With sufficient reserves, exiting 4e18 loose xPHAR returns 2e18 PHAR, which converts and deposits as 2e18 xPHAR, and a `compoundGross` of one base unit passes the input check while rounding the fee to zero. The gates are real, though: the returned PHAR must still clear the vault minimum, the conversion must be unpaused, the transfers must be permitted, and the treasury must actually hold the loose xPHAR. The staked position is not reachable this way, and a failed gate reverts the whole transaction. Loose xPHAR is not halved unconditionally.

On slippage, the backend requests a 5 percent limit from ODOS and 500 bps from KyberSwap (`server/automation.ts`), and the manual-compound flow quotes through the same server route, so the zero-minimum native builder found in the client sources does not describe the current manual flow. The narrower observation stands: the treasury contract itself enforces no fair-value output floor, and every bound that exists lives in calldata the same role produces. No honest-caller slippage exploit is established.

Location of Affected Code

File: [contracts/TreasuryV3.sol#L205-L232]

```
function autoCompound(
    uint256 compoundGross,
    address odosRouter,
    bytes calldata swapCalldata
) external onlyOwnerOrAutomator nonReentrant {
    require(autoCompoundEnabled, "Auto-compound not enabled");
    require(compoundGross > 0, "Amount must be greater than 0");
    require(odosRouter != address(0), "Invalid router address");

    uint256 rewardBal = IERC20(rewardAsset).balanceOf(address(this));
    require(rewardBal >= compoundGross, "Insufficient reward asset balance");

    uint256 feeAmount = (compoundGross * FEE_BPS) / 10000;
    uint256 swapAmount = compoundGross - feeAmount;
```

```
if (feeAmount > 0) {
  require(IERC20(rewardAsset).transfer(FEE_RECIPIENT, feeAmount), "Fee transfer failed");
}

IERC20(rewardAsset).approve(odosRouter, swapAmount);

uint256 pharBefore = IERC20(pharToken).balanceOf(address(this));
(bool swapSuccess, ) = odosRouter.call(swapCalldata);
require(swapSuccess, "ODOS swap failed");
IERC20(rewardAsset).approve(odosRouter, 0);

uint256 pharReceived = IERC20(pharToken).balanceOf(address(this)) - pharBefore;
require(pharReceived > 0, "No PHAR received from swap");
```

Impact

Within the confirmed trust model there is no unauthorized path, so the impact is disclosure: the trusted role's compounding authority includes choosing the swap target and calldata and the amount of held reward tokens spent, subject to the contract's balance and execution checks. The owner retains configuration control, direct withdrawals, and the ability to disable compounding or revoke the automator at any time. Router restrictions and independently enforced output bounds remain optional hardening under this model.

Recommendation

If the product ever wants to bound a misbehaving bot, the constraint has to be enforced independently of it. Reviewed adapters that restrict the target and validate the swap's input token, output token, recipient, and spend do that; a bare router allowlist does not, because it says nothing about the calldata that router accepts. An economic floor has the same requirement: a minimum freely selected by the bot cannot constrain the bot. Keep owner revocation and the compounding toggle available, and describe the role's actual powers in the whitepaper. Adopt a retained-allocation accumulator only if the product deliberately chooses allocation-based spending; the existing balance-based policy is not disproved by the unused latest-claim snapshot. Deployed instances have no upgrade path, so disabling compounding or revoking the automator is the operational control available today.

Team Response

Acknowledged.

I-08

INFO FIXED

One Wei of AVAX Permanently Locks a Contract-Owned Treasury Out of `fullWithdraw()`

Description

`receive()` takes AVAX from anyone, with no floor on the amount. `fullWithdraw()` ends by pushing the entire AVAX balance to `msg.sender` and reverts unless the push succeeds. An owner that is a contract without a payable fallback fails that push every time. Until someone funds the treasury, a zero balance lets `fullWithdraw()` skip the leg and complete normally. Once a third party puts a single wei in, the balance never returns to zero, so the final leg reverts from that block forward, and the whole bundled withdrawal reverts with it, token transfers and unstaking included. Ownership is immutable. There is no owner change and no partial mode inside `fullWithdraw()`.

TreasuryV4 does not change this. It inherits `receive()` and `fullWithdraw()` from TreasuryV3 unchanged and adds only `recoverToken()`, whose NatSpec states that native AVAX stays on the existing withdrawal functions (`contracts/TreasuryV4.sol`). Both versions carry the same behavior.

Whether anyone is exposed depends on who owns treasuries. Both factories deploy treasuries owned by their callers with no restriction to EOAs, so a contract-owned treasury is creatable. The team's clarification is that owners route claims and transfers to their own designated C-chain EVM address, and an on-chain check on 15 October 2026 found all sixteen then-current owners to be plain accounts; treat that count as a dated observation. Plain accounts accept native AVAX, and so do common contract wallets such as Safe multisigs, so the condition needs an owner contract that actively rejects native transfers, which falls outside the expected owner population. That is why this is informational. The Moat payout contract does not change the picture: as the payout wallet it receives only the ERC-20 claim payout, while `fullWithdraw()` pays AVAX to the owner, and `MoatRewardAutomation` accepts AVAX through its payable `receive()` regardless.

The scope of the damage, where it applies, is the bundled entry point: `withdrawAVAX()` sends to the same fixed recipient and fails the same way, so native value was already undeliverable for this owner class, while separate unstaking, conversion, PHAR withdrawal, and P33 withdrawal keep working subject to their own dependencies.

Two things that do not fix it. A `msg.value > 0` check in `receive()` stops the free zero-value event spam but not this attack, since one wei passes that check. Refusing native receipt entirely does not help either, because selfdestruct and coinbase forced transfers can fund the balance regardless.

Location of Affected Code

File: contracts/TreasuryV3.sol#L382-L384

```
receive() external payable {
  emit AVAXDeposited(msg.sender, msg.value);
}
```

File: contracts/TreasuryV3.sol#L322-L326

```
uint256 avaxBal = address(this).balance;
if (avaxBal > 0) {
  (bool success,) = payable(msg.sender).call{value: avaxBal}("");
  require(success, "AVAX transfer failed");
}
```

Impact

One wei plus gas disables the bundled withdrawal for any treasury whose owner rejects native transfers, for as long as the owner keeps rejecting. No principal class is lost; each still leaves through its dedicated function. The condition is permanent while ownership and the owner's receive behavior stay fixed. Under the team's clarification that owners are their own wallet addresses, no expected deployment is affected.

Recommendation

Document that a treasury owner must be able to receive native AVAX. If contract owners are ever supported, let the owner designate an authorized payable recipient for native withdrawals, or split native egress out of the bundled path so its failure cannot revert unrelated principal transfers. TreasuryV4 does not include either change, so it would need the corrected egress too; existing non-upgradeable instances need an operational workaround for this owner class.

Team Response

Fixed.



Your security partner, for the long term

SHIELDIFY.ORG · X.COM/SHIELDIFYSEC · LINKEDIN